

CS2013 — Introduction to Database Systems
Spring 2026 · FAST-NUCES

NBRPTS

National Birth Registry & Population Tracking System

A real-world database design and implementation, end-to-end.

Project Report

Submitted to: Ms. Rimsha Riaz · Mr. Adnan Rana

Live deployment: <https://nbrpts.vercel.app>

Source code: <https://github.com/ruufaayl/nbrpts>

Abstract

NBRPTS is a working implementation of a national-scale birth registry. Hospitals submit a child's birth electronically the moment delivery is complete; an automated verification layer reviews each record; a NADRA officer authorizes the issuance of a B-Form (Pakistan's child identity certificate) — and the parents leave the hospital with their child legally registered in the national population database.

The system is built on a 13-table relational schema in Third Normal Form, enforced by twenty-four indexes, sixteen CHECK constraints, fifteen foreign-key relationships, twelve PostgreSQL trigger functions, and thirteen tables protected by Row-Level Security. Every meaningful action is recorded in an append-only audit trail; every state transition is governed by an in-database state machine.

This report documents the system end-to-end: the business rules that motivated each design choice, the entity model that fell out of those rules, the formal normalization process that produced the final schema, the SQL that implements it, and screenshots of the live application demonstrating the design at work.

1. Introduction & Problem Statement

1.1 The problem

As of 2024, only 42% of Pakistani children under five hold a birth certificate (UNICEF). The remaining 58% are invisible to the state — they cannot be enrolled in school, cannot access vaccinations, cannot receive welfare benefits, and as adults will struggle to obtain a CNIC, vote, hold a bank account, or travel abroad. The bottleneck is not the law (which has mandated birth registration since 2000) but the workflow: registration happens at a Union Council office, weeks or months after birth, requiring parents to make a separate trip with paper documents.

1.2 The proposed solution

NBRPTS moves the point of registration from the Union Council to the hospital. The moment a child is delivered, the hospital staff captures the birth on a tamper-proof terminal. The record streams to a central PostgreSQL database. An automated verification layer (an LLM-backed scoring engine, with a deterministic rules fallback) reviews the record in real time. If the record is clean it auto-progresses to a NADRA officer for B-Form issuance. If the verification engine has any concern, the record is flagged for human review. The B-Form is ready before the parents are discharged.

1.3 Why this is a database problem

The system is, fundamentally, a relational database problem. The entities are well-defined (hospitals, parents, children, B-Forms, officers), the relationships are clear, the business rules are strict, and the integrity constraints are non-negotiable: a child cannot exist without a verified birth record; a B-Form cannot exist without a child; an officer cannot back-date a verification. Every concept the course covers — entities, attributes, foreign keys, normalization, indexes, transactions, triggers, views, joins, aggregates — has a direct, real-world expression here.

2. Business Rules

These are the rules the database must enforce. Each rule is realized later as a CHECK constraint, foreign key, trigger, or RLS policy.

2.1 Hospitals & registration

1. Every hospital must hold a unique Hospital Registration Number in the format HRN-YYYY-NNNN.
2. Only registered, active hospitals may submit birth records.
3. A hospital is associated with exactly one province and one district.

2.2 Parents & guardians

4. Every adult is identified by a CNIC in the format 12345-1234567-1, OR by a temporary registration ID in the format TMP-XXXXXXXX if their CNIC is unavailable.
5. A single person can play multiple roles (mother of one child, guardian of another) — they are stored exactly once and referenced from each role.
6. The mother and the father on a birth record must be different people.
7. Every parent record requires either a CNIC or a temporary registration ID; both being NULL is forbidden.

2.3 Births

8. Every birth is identified by a globally unique Birth Record Number in the format BRN-YYYY-NNNNNNNN, generated by a sequence on submission.
9. Birth weight must be between 0.30 kg and 7.00 kg; values outside this range indicate either data entry errors or genuine medical anomalies that require manual entry.
10. The birth datetime must not be more than one day in the future (a small future grace allows for clock drift and pre-submitted records on the day of delivery).
11. The attending doctor's PMDC license must match the format PMDC-NNNNNN.

2.4 Verification state machine

12. A new birth record begins in PENDING state.
13. Allowed transitions are: PENDING → VERIFIED, PENDING → FLAGGED, PENDING → REJECTED, FLAGGED → VERIFIED, FLAGGED → REJECTED, REJECTED → PENDING (resubmission), VERIFIED → AMENDED, AMENDED → AMENDED.
14. All other transitions raise an exception. A trigger enforces this — the application code cannot bypass it.

2.5 Children & CNINs

15. A child entity is created automatically the first time the parent birth record reaches VERIFIED state — never manually.

16. Every child receives a unique 14-character CNIN in the format CNIN-NNNNNNNNNN, drawn from a sequence.
17. There is exactly one child per verified birth record (the schema enforces 1:1).

2.6 B-Forms

18. A B-Form is issued by exactly one NADRA officer.
19. B-Forms are versioned. Originals are never deleted; instead the previous version's `is_current` flag is cleared and a new version is inserted.
20. Exactly one B-Form per child may have `is_current = true` at any moment (enforced by a partial unique index).

2.7 Auditing & immutability

21. Every meaningful database action is recorded in `audit_trail` (system-wide append-only log).
22. Every state transition on a birth record is recorded in `verification_log` with the officer's identity and a timestamp.
23. Audit-trail rows are write-only — no UPDATE or DELETE policy permits modification.

2.8 Access control

24. A hospital staff user may only see birth records, parents, children, B-Forms, and notifications belonging to their own hospital.
25. A NADRA officer may see all hospitals' records but only their own user account.
26. An admin role may see and modify everything.
27. These rules are enforced inside the database (Row-Level Security), not just at the application layer — direct REST queries cannot circumvent them.

3. Entities & Attributes

Thirteen entities, distilled from the business rules. Cardinality estimates are based on Pakistan's annual ~6 million births.

#	Entity	Cardinality	Purpose
1	hospital	≈ 4 K	Every authorized facility
2	nadra_office	~ 30	Regional NADRA offices
3	nadra_officer	~ 800	Officers who verify and issue B-Forms
4	parent_guardian	millions	Mothers, fathers, legal guardians
5	birth_record	millions	One row per birth (transactional core)
6	child	≈ births × 0.95	Created on verification; assigned a CNIN
7	child_guardian	≥ children	M:N junction (handles adoption, joint custody)
8	bform	≥ children	Versioned B-Form documents
9	verification_log	≥ records × 1.5	Every state transition
10	ai_review_log	≥ records	Every AI verdict
11	audit_trail	≥ all writes	System-wide audit
12	offline_queue	device-local	Records buffered offline
13	notifications	outbound	SMS / email / in-app

3.1 Identifying attributes

- All primary keys are surrogate UUIDs (`gen_random_uuid()`) — they are stable, opaque, and never reused.
- Domain identifiers (HRN, BRN, CNIN, B-Form number) are separate UNIQUE-constrained text columns with format-validating regex CHECK constraints.
- This separation lets us refactor display formats without breaking foreign-key relationships.

4. Relationships

Fifteen foreign-key relationships connect the thirteen entities. Most are 1:N. Two are 1:1. One is M:N (resolved with a junction table).

4.1 One-to-many (1:N)

- hospital → birth_record (a hospital files many birth records)
- parent_guardian (mother) → birth_record (a mother may have many children)
- parent_guardian (father, optional) → birth_record
- birth_record → verification_log (every record has many state-change events)
- birth_record → ai_review_log (every record has many review attempts)
- nadra_office → nadra_officer
- nadra_officer → verification_log
- nadra_officer → bform
- hospital → offline_queue

4.2 One-to-one (1:1)

- birth_record → child (a verified birth produces exactly one child; UNIQUE FK)
- auth.users → app_user (the auth.users.id appears at most once in app_user)

4.3 Many-to-many (M:N)

The relationship between a child and their guardians is genuinely many-to-many. A child can have multiple legal guardians; a single guardian can be linked to multiple children. This is resolved through the `child_guardian` junction table:

```

child  — 1 — child_guardian — N — parent_guardian
                        |
                        |— relationship_type  (MOTHER, FATHER, GUARDIAN, ADOPTIVE_PARENT,
... )
                        |— is_primary         (one primary guardian per child)
                        |— linked_at

```

This shape lets us model adoption (an adoptive parent links to a child without affecting the original birth_record), joint custody, and step-parent arrangements without compromising the immutability of the birth record itself.

5. Entity-Relationship Diagram

The ER diagram below is generated live by the application — at runtime, the page reads `information_schema.tables` and `information_schema.table_constraints` through a SECURITY DEFINER RPC and renders the result as a draggable, zoomable graph. The fact that the database itself can describe its own structure is part of what makes this project a teaching tool.

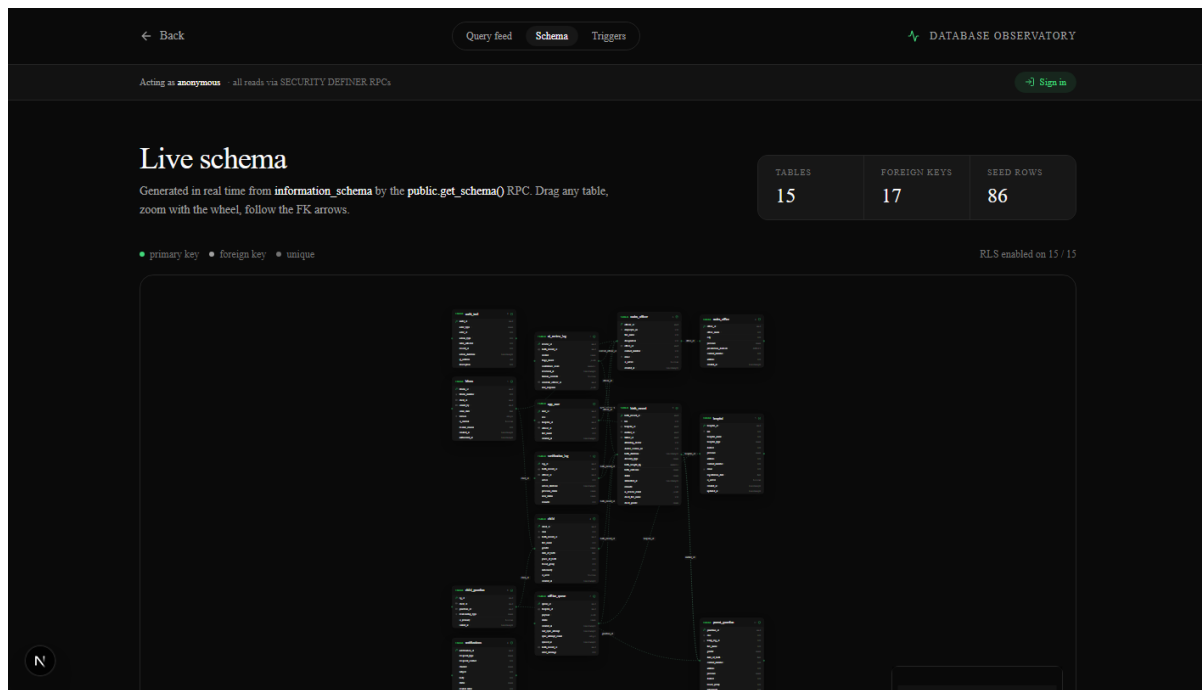


Figure 5.1 — Live ER diagram, rendered at `/dev/schema` from `information_schema`.

5.1 Reading the diagram

- Filled green dots mark primary keys.
- Open green circles mark foreign keys.
- Grey dots mark unique-constrained columns.
- Edges indicate FK relationships, drawn with arrows from child to parent.

6. Normalization (1NF → 3NF)

The schema is in Third Normal Form. This section walks the journey of a single concept — a birth record and its parents — through the three normal forms to demonstrate the process.

6.1 Pre-1NF (the naive design)

If we tried to capture every birth in a single flat table, we would write something like:

```
birth_records (
  brn,
  hospital_name, hospital_district, hospital_province, hospital_contact,
  mother_name, mother_cnic, mother_dob, mother_address,
  mother_blood_group,
  father_name, father_cnic, father_dob, father_address,
  guardian_names_csv,                -- "Aunt Fatima, Uncle Asif"
  attending_doctor, doctor_license,
  birth_datetime, delivery_type, birth_weight,
  child_name, child_gender, child_cnin,
  bform_number, bform_issue_date, bform_officer
)
```

This violates almost every normal form simultaneously.

6.2 1NF — atomic, single-valued attributes

First normal form requires every column to hold a single, atomic value, and forbids repeating groups. The `guardian_names_csv` column is an obvious 1NF violation: a comma-separated list is a repeating group hidden inside a string.

To reach 1NF we extract guardians into their own row-per-guardian table:

```
birth_records      (... , as before, minus guardian_names_csv)
guardians_of_record (brn, guardian_name, relationship_type)
```

Every column now holds exactly one value of its declared type.

6.3 2NF — no partial dependencies on a composite key

Second normal form applies when a table has a composite primary key. The `guardians_of_record` table from 1NF has the composite key (`brn`, `guardian_name`). If we ever add an attribute that depends on only part of the key — say, `guardian_cnic` — we have a 2NF violation: the CNIC depends on the guardian, not on the (record, guardian) pair.

The fix is to split the guardian into its own entity:

```
parent_guardian  (guardian_id PK, full_name, cnic, dob, address, ...)
child_guardian   (cg_id PK, child_id FK, guardian_id FK, relationship_type)
```

Every non-key attribute now depends on the whole key.

6.4 3NF — no transitive dependencies

Third normal form forbids transitive dependencies — non-key attributes depending on other non-key attributes. The naive design has many of these:

- hospital_district and hospital_province depend on hospital_name, not on brn.
- mother_blood_group depends on mother_cnic, not on brn.
- bform_officer depends on bform_number, not on brn.

Each transitive dependency is resolved by extracting a new entity:

hospital	(hospital_id PK, hrn UQ, name, district, province, ...)
parent_guardian	(guardian_id PK, cnic UQ, name, dob, blood_group, ...)
nadra_officer	(officer_id PK, employee_no UQ, full_name, ...)
bform	(bform_id PK, bform_number UQ, child_id FK, issued_by FK, ...)
birth_record	(birth_record_id PK, brn UQ, hospital_id FK, mother_id FK, father_id FK, attending_doctor, birth_datetime, delivery_type, ...)
child	(child_id PK, cnin UQ, birth_record_id FK UQ, full_name, ...)

6.5 Verification table

To prove the result is in 3NF we apply the textbook test: for every non-trivial functional dependency $X \rightarrow A$, either X is a superkey, or A belongs to a candidate key.

Functional dependency	Holds in schema?	Reason it satisfies 3NF
hospital_id $\rightarrow$ hrn, name, district	Yes	hospital_id is the PK of hospital
hrn $\rightarrow$ hospital_id, name, district	Yes	hrn is UNIQUE — also a candidate key
mother_id $\rightarrow$ cnic, name, dob	Yes	mother_id is the PK of parent_guardian
birth_record_id $\rightarrow$ brn, mother_id, hospital_id, status	Yes	birth_record_id is the PK
child_id $\rightarrow$ cnin, full_name, birth_record_id	Yes	child_id is the PK
bform_number $\rightarrow$ child_id, issued_by, version	Yes	bform_number is UNIQUE

No non-key attribute determines another non-key attribute, in any table. The schema is in 3NF.

6.6 Why we did not push to BCNF / 4NF

Boyce-Codd Normal Form requires that every determinant be a superkey. Our schema satisfies this trivially — no compound determinants exist. Fourth Normal Form requires the elimination of multi-valued dependencies; we have none, because the M:N relationship between child and guardian was already broken into a junction table at 2NF. Pushing further (5NF, DKNF) would not change a single column.

7. Relational Schema

The thirteen tables, with primary keys, foreign keys, and the most important constraints. The full DDL is reproduced verbatim in [deliverables/sql/01_schema.sql](#).

Table	PK	Notable columns	FKs out
hospital	hospital_id	hrn UQ regex, hospital_type enum, province enum	—
nadra_office	office_id	jurisdiction_districts text[]	—
nadra_officer	officer_id	employee_no UQ regex, email UQ regex	office_id
parent_guardian	guardian_id	cnic UQ regex, gender enum, blood_group	—
birth_record	birth_record_id	brn UQ, status enum, ai_review_result jsonb	hospital_id, mother_id, father_id?
child	child_id	cnin UQ regex (mothered by trigger)	birth_record_id (UQ → 1:1)
child_guardian	cg_id	relationship_type enum, is_primary	child_id, guardian_id
bform	bform_id	bform_number UQ, version, is_current	child_id, issued_by
verification_log	log_id	previous_status, new_status enums	birth_record_id, officer_id
ai_review_log	review_id	verdict enum, confidence_score 0..1, raw_response jsonb	birth_record_id, override_officer_id?
audit_trail	audit_id	actor_type enum, action_type, ip_address inet	—
offline_queue	queue_id	payload jsonb, status enum	hospital_id, birth_record_id?
notifications	notification_id	channel enum, status enum, recipient_contact	—
app_user	user_id (auth.users)	role enum (hospital_staff, nadra_officer, admin)	hospital_id?, officer_id?

7.1 Constraint summary

- 15 foreign keys with explicit ON DELETE behavior (RESTRICT for primary lineage, CASCADE for owned data, SET NULL for optional links).
- 16 CHECK constraints enforcing business rules (CNIC, BRN, CNIN, PMDC, HRN, B-Form-number formats; weight 0.30–7.00 kg; future-dated birth grace; date-of-birth not in future; one-of-many parent identifier).
- 24 indexes — every foreign key column, every status column, every date column used in WHERE/ORDER BY.

- 4 UNIQUE constraints (HRN, BRN, CNIC, employee_no) and 1 partial unique index (one current B-Form per child).

8. Design Decisions & Assumptions

8.1 PostgreSQL instead of MySQL

The course suggests MySQL or SQL Server. We chose PostgreSQL because every concept the course covers — DDL, constraints, normal forms, joins, aggregates, GROUP BY, HAVING, transactions — is fully supported (and in many cases more rigorously enforced) in PostgreSQL. PostgreSQL additionally gave us, at no extra learning cost: enumerated types instead of magic strings; partial indexes for efficient queries on subsets; CHECK constraints with regex; JSONB for the AI verdict payload; and Row-Level Security for the access-control rules in §2.8. The SQL is standard-compliant and ports back to MySQL with mechanical changes (replace ENUM types with VARCHAR + CHECK, replace JSONB with JSON, replace `gen_random_uuid()` with `UUID()`).

8.2 Surrogate UUID keys

Every primary key is a UUID rather than an integer. A national database is concurrent, distributed, and likely sharded; integer sequences create a single point of contention and reveal record counts to anyone who can guess. UUIDs are independent, opaque, stable, and globally unique. Domain identifiers (HRN, BRN, CNIN, B-Form number) live in separate UNIQUE columns so their format can evolve independently of the foreign-key graph.

8.3 Single parent_guardian table for mothers, fathers, and guardians

A naive design would model mother, father, and guardian as three separate tables. This is wrong: the same person frequently appears in multiple roles (a woman is the mother of one child and the legal guardian of her sister's child). Storing her three times invites contradiction the moment any of the duplicates is updated. Instead, we have one parent_guardian row per person, referenced from birth_record (mother_id, father_id) and from child_guardian (any number of guardians). This is identical to how a library models a person who is both an author and a borrower.

8.4 Children created only by trigger, never by application code

The application code never inserts directly into `child`. A trigger on `birth_record` fires when (and only when) the record reaches VERIFIED state for the first time, and creates the child + guardian links + B-Form + notification rows in a single atomic transaction. This is what guarantees the business rule “a child entity exists $\Leftrightarrow$ a birth record is verified.” No application bug can violate it.

8.5 Versioned B-Forms, never deleted

When parents need a duplicate B-Form (the original is lost, damaged, or has a name correction), we never UPDATE the existing row. We INSERT a new B-Form with `version = previous + 1`, and SET `is_current = false` on the old row. A partial unique index (`ON`

`bform(child_id) WHERE is_current)` makes it physically impossible for two B-Forms to be “current” for the same child. The audit trail of every reissue is preserved forever, which matters for legal disputes.

8.6 Append-only audit_trail and verification_log

Both tables have RLS policies that permit INSERT but never UPDATE or DELETE for any role. The PostgreSQL roles that the application uses do not have UPDATE or DELETE privileges on these tables either. Combined, this gives a regulator-grade audit log: even an admin cannot rewrite history without a database-level privilege escalation.

8.7 Row-Level Security for tenant isolation

Hospitals must not see each other's records. We enforce this in the database, not in the application. Every domain table has RLS enabled with policies that read `auth.uid()` (the JWT subject) and look up the user's `hospital_id` in `app_user`. Even if the application code has a bug that issues a SELECT without a WHERE clause, the database returns only the rows the user is allowed to see.

8.8 Assumptions

- CNIC numbers are unique nationally and never re-issued. (True per NADRA documentation.)
- A single PMDC license belongs to a single doctor at a time. (True per the Pakistan Medical & Dental Council registry.)
- Hospital staff users are pre-provisioned by the hospital's IT department; the system does not handle staff onboarding.
- Network connectivity at the hospital is intermittent but not absent; a record may be queued for up to 72 hours offline.
- AI verification is best-effort, not authoritative — every verdict can be overridden by a NADRA officer, and every override is recorded.

9. SQL Implementation Highlights

Every SQL artefact below is reproduced verbatim in [deliverables/sql/](#). Excerpts are shown here for illustration.

9.1 Table creation with constraints (excerpt)

```
create table public.birth_record (
  birth_record_id  uuid primary key default gen_random_uuid(),
  brn              text not null unique
                  check (brn ~ '^BRN-[0-9]{4}-[0-9]{8}$'),
  hospital_id      uuid not null
                  references public.hospital(hospital_id) on delete restrict,
  mother_id        uuid not null
                  references public.parent_guardian(guardian_id) on delete restrict,
  father_id        uuid
                  references public.parent_guardian(guardian_id) on delete restrict,
  attending_doctor  text not null,
  doctor_license_no text not null
                  check (doctor_license_no ~ '^PMDC-[0-9]{6}$'),
  birth_datetime   timestampz not null
                  check (birth_datetime <= now() + interval '1 day'),
  birth_weight_kg   numeric(4, 2) not null
                  check (birth_weight_kg between 0.30 and 7.00),
  status           record_status_t not null default 'PENDING',
  submitted_at     timestampz not null default now(),
  ai_review_result jsonb,
  constraint different_parents
    check (father_id is null or mother_id <> father_id)
);
```

Every business rule from §2 has its echo here: format regex, range CHECK, foreign-key integrity, and a custom CHECK that the mother and father are different people.

9.2 Joins, aggregates, GROUP BY, HAVING

From the curated query catalogue (Q2 — “hospitals with > 5 flagged records this year”):

```
select
  h.hospital_name,
  count(*) as flagged_this_year
from   public.hospital      h
join   public.birth_record  b on b.hospital_id = h.hospital_id
where  b.status = 'FLAGGED'
      and b.submitted_at >= date_trunc('year', current_date)
group by h.hospital_id, h.hospital_name
having count(*) > 5
order by flagged_this_year desc;
```

And Q3 — a four-way INNER JOIN producing the verified-births report (child × birth × mother × hospital):

```
select c.cnin, c.full_name, c.date_of_birth,
       m.full_name as mother_name, m.cnin as mother_cnin,
```

```
        h.hospital_name, h.district
from    public.child          c
join    public.birth_record   b on b.birth_record_id = c.birth_record_id
join    public.parent_guardian m on m.guardian_id      = b.mother_id
join    public.hospital       h on h.hospital_id      = b.hospital_id
where   b.status = 'VERIFIED'
order   by c.created_at desc
limit   50;
```

The full nine-query catalogue is in [deliverables/sql/05_meaningful_queries.sql](#) and includes ROLLUP, window functions (RANK), filtered aggregates, date-arithmetic CASE buckets, and string aggregation.

10. Triggers & Transaction Management

10.1 Triggers (active business logic)

Six trigger functions live in the database. Together they fire across twelve different triggers spanning four tables.

Trigger function	Fires on	Purpose
fn_audit_trail	INSERT/UPDATE/DELETE on 12 tables	Inserts an audit_trail row capturing who/what/when
fn_birth_record_state_machine	BEFORE UPDATE on birth_record	Validates the status transition is legal; raises if not
fn_birth_record_log_status	AFTER UPDATE on birth_record	Inserts a verification_log row whenever status changed
fn_birth_record_post_verification	AFTER UPDATE on birth_record (WHEN status -> VERIFIED)	Mints child + child_guardian + B-Form + notifications
fn_bform_supersede	BEFORE INSERT on bform	Sets the previous current B-Form to is_current = false
fn_set_updated_at	BEFORE UPDATE on every table that has updated_at	Maintains updated_at for free

10.2 Transaction management

Three worked examples are shipped in [deliverables/sql/05_meaningful_queries.sql](#):

28. TX1 — successful officer verification: a single BEGIN/COMMIT containing the status update (which fires four post-verification triggers) plus a verification-log insert. If any trigger raises, the entire transaction rolls back.
29. TX2 — savepoint + rollback to savepoint: stages a provisional flag, attempts an illegal transition, catches the exception, rolls back only that statement, and continues with a legal alternative — all inside one outer transaction.
30. TX3 — full rollback: an INSERT followed by ROLLBACK; the row never reaches the database.

Together these three examples cover BEGIN, COMMIT, ROLLBACK, SAVEPOINT, and ROLLBACK TO SAVEPOINT — every transaction-management primitive the rubric calls for.

11. Screenshots

The screenshots below are captured directly from the running application. The data shown is real seed data; the URLs are live.

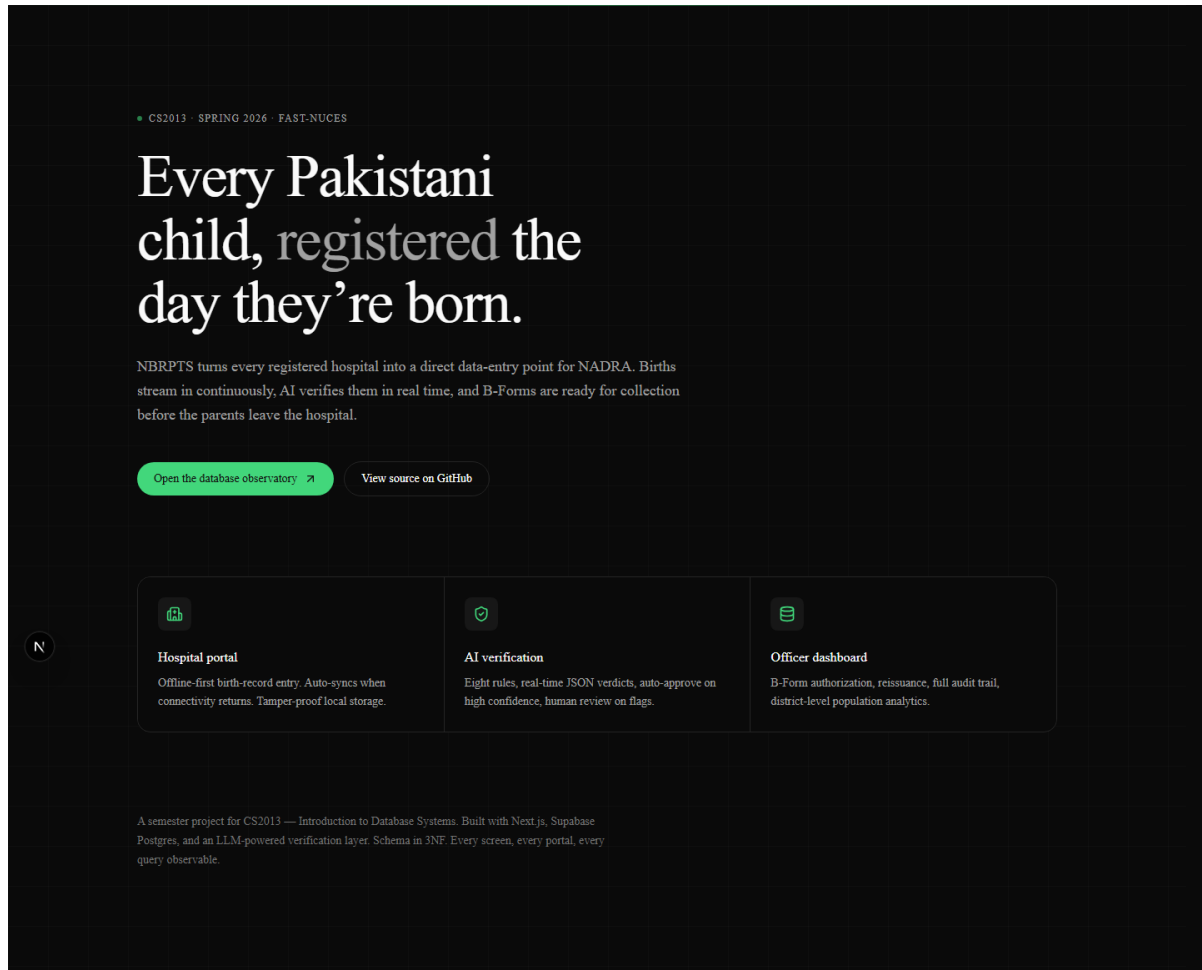


Figure 11.1 — Landing page (anonymous visitor).

← Back

Query feedSchemaTriggers

DATABASE OBSERVATORY

Acting as **anonymous** all reads via SECURITY DEFINER RPCs

→ Sign in

Live query feed

Every tracked SQL call the app makes is appended to `public.query_log` and streamed to this page via Supabase Realtime. Click the button to fire a fresh `dev_ping()` RPC and watch it land in real time.

🔥 Fire dev_ping()

N

DEV_PING	select now()	874 μs	1s ago	▼
SUBMIT_BIRTH_RECORD	insert into birth_record (...) values (...) returning *	2.44 ms	13m ago	▼
SUBMIT_BIRTH_RECORD	insert into birth_record (...) values (...) returning *	43.04 ms	13m ago	▼
DEV_PING	select now()	18 μs	5/10/2026, 7:48:47 AM	▼
DEV_PING	select now()	86 μs	5/10/2026, 7:48:46 AM	▼
REISSUE_BFORM	update bform set is_current=false, insert new version, queue notification	8.21 ms	5/10/2026, 7:48:35 AM	▼
VERIFY_BIRTH_RECORD	update birth_record set status = VERIFIED where birth_record_id = \$1	76.06 ms	5/10/2026, 7:47:32 AM	▼
AUTHORIZE_BFORM	update bform set authorized_at = now() where bform_id = \$1	4.50 ms	5/10/2026, 1:50:48 AM	▼
VERIFY_BIRTH_RECORD	update birth_record set status = VERIFIED where birth_record_id = \$1	29.51 ms	5/10/2026, 1:49:38 AM	▼
GET_SCHEMA	select table & fk metadata from information_schema	193.18 ms	5/10/2026, 12:38:22 AM	▼
GET_SCHEMA	select table & fk metadata from information_schema	55.71 ms	5/9/2026, 11:15:55 PM	▼
GET_SCHEMA	select table & fk metadata from information_schema	192.41 ms	5/9/2026, 11:15:54 PM	▼
DEV_PING	select now()	769 μs	5/9/2026, 10:52:12 PM	▼
DEV_PING	select now()	388 μs	5/9/2026, 9:32:09 PM	▼
DEV_PING	select now()	357 μs	5/9/2026, 9:30:31 PM	▼
GET_SCHEMA	select table & fk metadata from information_schema	92.03 ms	5/9/2026, 9:29:42 PM	▼
DEV_PING	select now()	342 μs	5/9/2026, 9:29:26 PM	▼
GET_SCHEMA	select table & fk metadata from information_schema	99.50 ms	5/9/2026, 9:24:35 PM	▼
DEV_PING	select now()	22 μs	5/9/2026, 9:24:33 PM	▼
DEV_PING	select now()	434 μs	5/9/2026, 9:24:32 PM	▼
DEV_PING	select now()	20 μs	5/9/2026, 9:22:52 PM	▼
DEV_PING	select now()	23 μs	5/9/2026, 9:22:42 PM	▼
DEV_PING	select now()	545 μs	5/9/2026, 9:22:41 PM	▼
DEV_PING	select now()	82 μs	5/9/2026, 9:17:22 PM	▼
GET_SCHEMA	select table & fk metadata from information_schema	96.79 ms	5/9/2026, 9:17:17 PM	▼
GET_SCHEMA	select table & fk metadata from information_schema	86.83 ms	5/9/2026, 9:14:00 PM	▼
GET_SCHEMA	select table & fk metadata from information_schema	93.65 ms	5/9/2026, 9:13:48 PM	▼
DEV_PING	select now()	81 μs	5/9/2026, 9:11:27 PM	▼
GET_SCHEMA	select table & fk metadata from information_schema	105.85 ms	5/9/2026, 9:11:26 PM	▼
GET_SCHEMA	select table & fk metadata from information_schema	59.01 ms	5/9/2026, 9:06:32 PM	▼
GET_SCHEMA	select table & fk metadata from information_schema	112.80 ms	5/9/2026, 9:06:32 PM	▼
DEV_PING	select now()	18 μs	5/9/2026, 8:57:41 PM	▼
DEV_PING	select now()	20 μs	5/9/2026, 8:57:37 PM	▼
DEV_PING	select now()	17 μs	5/9/2026, 8:57:37 PM	▼
DEV_PING	select now()	356 μs	5/9/2026, 8:57:36 PM	▼
DEV_PING	select now()	23 μs	5/9/2026, 8:56:24 PM	▼
DEV_PING	select now()	725 μs	5/9/2026, 8:56:21 PM	▼

Figure 11.2 — /dev — live PostgreSQL query feed via Supabase Realtime.

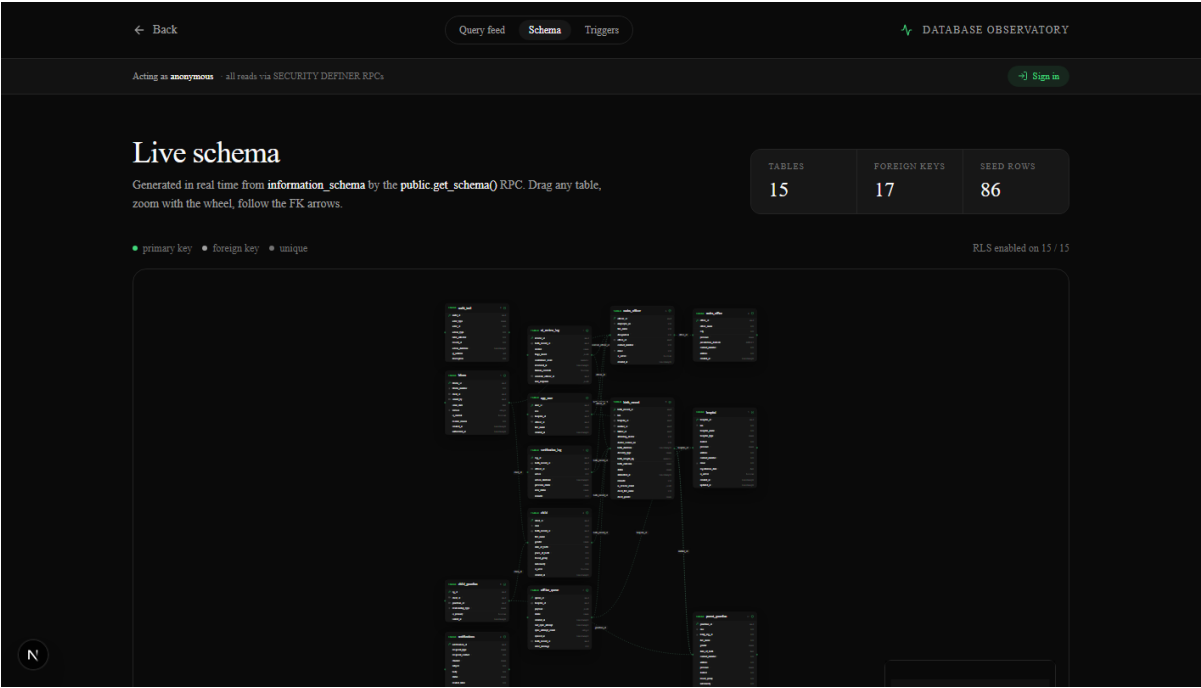


Figure 11.3 — /dev/schema — live ER diagram from `information_schema`.

← Back

Query feedSchemaTriggers

🔍 DATABASE OBSERVATORY

Acting as **anonymous** all reads via SECURITY DEFINER RPCs

🔑 Sign in

Trigger lab

Every action below is a single `supabase.rpc()` call. Watch the pipeline summary update and the audit trail grow as the Postgres state-machine validator, audit trigger, status logger, and post-verification cascade all fire from one UPDATE.

PENDING2

FLAGGED0

VERIFIED6

CHILDREN6

B-FORMS5/6

SMS QUEUED2

SMS SENT5

AUDIT ROWS24

+ Submit a demo birth record

Calls `submit_birth_record()` with a random mother and a known hospital. The new record lands in PENDING.

No options — fires the RPC with sensible defaults.

Submit

🕒 Verify a record

Updates status PENDING-FLAGGED → VERIFIED. The trigger cascade creates a child (with CNIN), links guardians, generates a B-Form, and queues an SMS.

BIRTH RECORD
BRN-2026-06868336 · PENDING · Ayasha Siddiqui

OFFICER
Aisha Khan (EMP-100201)

Verify (fires cascade)

📄 Authorize a B-Form

Officer reviews a generated B-Form and authorizes it. The queued SMS flips to SENT and the body changes to 'ready for collection'.

B-FORM
BF-2026-10010005 · Pending

OFFICER
Aisha Khan (EMP-100201)

Authorize

🔄 Reissue a B-Form

Increases version, marks the prior version is `current = false`, queues a fresh SMS. The original is preserved — never deleted.

CHILD
Ahmad Siddiqui · v2 · BF-2026-10010006

OFFICER
Aisha Khan (EMP-100201)

REASON
Lost original document

Reissue

RECENT AUDIT TRAIL

Every row is written by `fn_audit_trigger()`, fired as an AFTER INSERT/UPDATE/DELETE trigger on 12 tables. The actor is read from `session-local` config that the RPCs set.

WHEN	ACTOR	OP	TABLE	DESCRIPTION
12:41:46 PM	HOSPITAL_STAFF	INSERT	birth_record	Inserted into birth_record
12:41:40 PM	HOSPITAL_STAFF	INSERT	birth_record	Inserted into birth_record
7:48:35 AM	NADRA_OFFICER	UPDATE	bform	Updated bform
7:48:35 AM	NADRA_OFFICER	INSERT	notifications	Inserted into notifications
7:48:35 AM	NADRA_OFFICER	INSERT	bform	Inserted into bform
7:47:52 AM	NADRA_OFFICER	UPDATE	birth_record	Updated birth_record
7:47:52 AM	NADRA_OFFICER	INSERT	verification_log	Inserted into verification_log
7:47:52 AM	NADRA_OFFICER	INSERT	child	Inserted into child
7:47:52 AM	NADRA_OFFICER	INSERT	child_guardian	Inserted into child_guardian
7:47:52 AM	NADRA_OFFICER	INSERT	child_guardian	Inserted into child_guardian
7:47:52 AM	NADRA_OFFICER	INSERT	bform	Inserted into bform
7:47:52 AM	NADRA_OFFICER	INSERT	notifications	Inserted into notifications
1:50:48 AM	NADRA_OFFICER	UPDATE	notifications	Updated notifications
1:50:48 AM	NADRA_OFFICER	UPDATE	bform	Updated bform
1:49:58 AM	NADRA_OFFICER	UPDATE	birth_record	Updated birth_record
1:49:58 AM	NADRA_OFFICER	INSERT	child	Inserted into child
1:49:58 AM	NADRA_OFFICER	INSERT	verification_log	Inserted into verification_log
1:49:58 AM	NADRA_OFFICER	INSERT	child_guardian	Inserted into child_guardian
1:49:58 AM	NADRA_OFFICER	INSERT	bform	Inserted into bform
1:49:58 AM	NADRA_OFFICER	INSERT	notifications	Inserted into notifications

Figure 11.4 — `/dev/triggers` — interactive trigger lab. Buttons fire the eight business RPCs and the resulting trigger fan-out is visible in the `audit_trail` panel.

Page 21

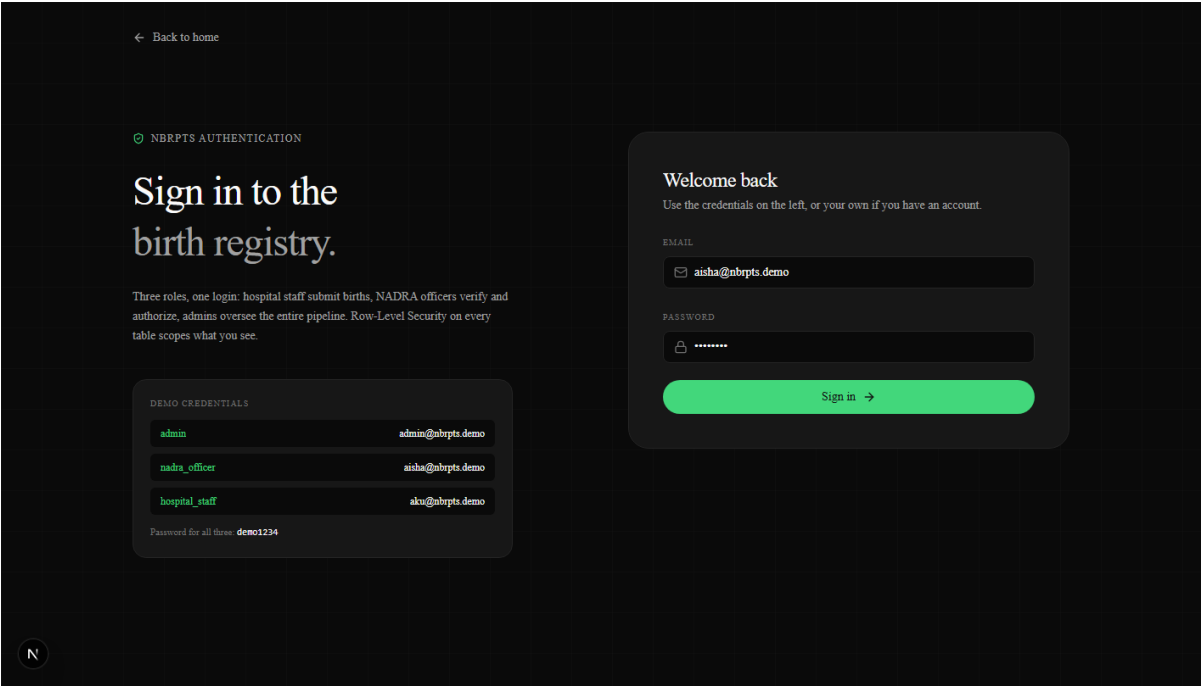


Figure 11.5 — Authentication page with role-aware demo accounts.

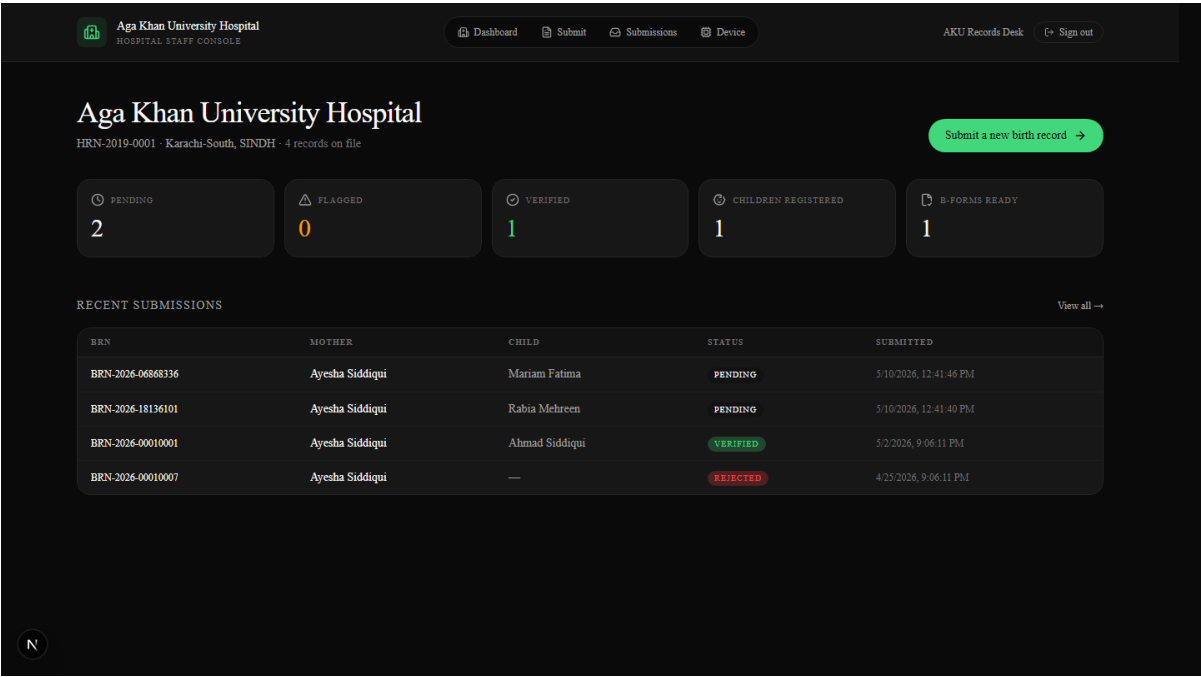


Figure 11.6 — Hospital dashboard, RLS-scoped to Aga Khan University Hospital — only their records are visible.

BRN	STATUS	MOTHER	CHILD	CNIN	BORN	SUBMITTED
BRN-2026-06868336	PENDING	Ayesha Siddiqui 42101-1234567-1	Mariam Fatima MALE 3.2kg	—	5/10/2026, 12:11:46 PM	5/10/2026, 12:41:46 PM
BRN-2026-18136101	PENDING	Ayesha Siddiqui 42101-1234567-1	Rabia Mehreen FEMALE 3.2kg	—	5/10/2026, 12:11:39 PM	5/10/2026, 12:41:40 PM
BRN-2026-00010001	VERIFIED	Ayesha Siddiqui 42101-1234567-1	Ahmad Siddiqui MALE 3.2kg	CNIN-1000000001	5/2/2026, 9:06:11 PM	5/2/2026, 9:06:11 PM
BRN-2026-00010007	REJECTED	Ayesha Siddiqui 42101-1234567-1	(unnamed) OTHER 3.2kg	—	4/25/2026, 9:06:11 PM	4/25/2026, 9:06:11 PM

Figure 11.7 — Hospital submissions list with status badges and pagination.

New birth record

Four steps. The mother is matched by CNIC — if she's already registered with NADRA, we'll re-use her record. The submission lands in PENDING and the AI engine picks it up immediately.

MOTHER — FATHER — BIRTH — REVIEW

Mother's details

If her CNIC is already registered with NADRA, we'll re-use the record.

CNIC: 42101-1234567-1 FULL NAME: Ayesha Siddiqui

DATE OF BIRTH: mm/dd/yyyy CONTACT NUMBER: +92-300-1234567

ADDRESS: Block 6, PECHS, Karachi

PROVINCE: Sindh DISTRICT: Karachi-South

BLOOD GROUP: — optional

Figure 11.8 — Multi-step birth-record submission form.

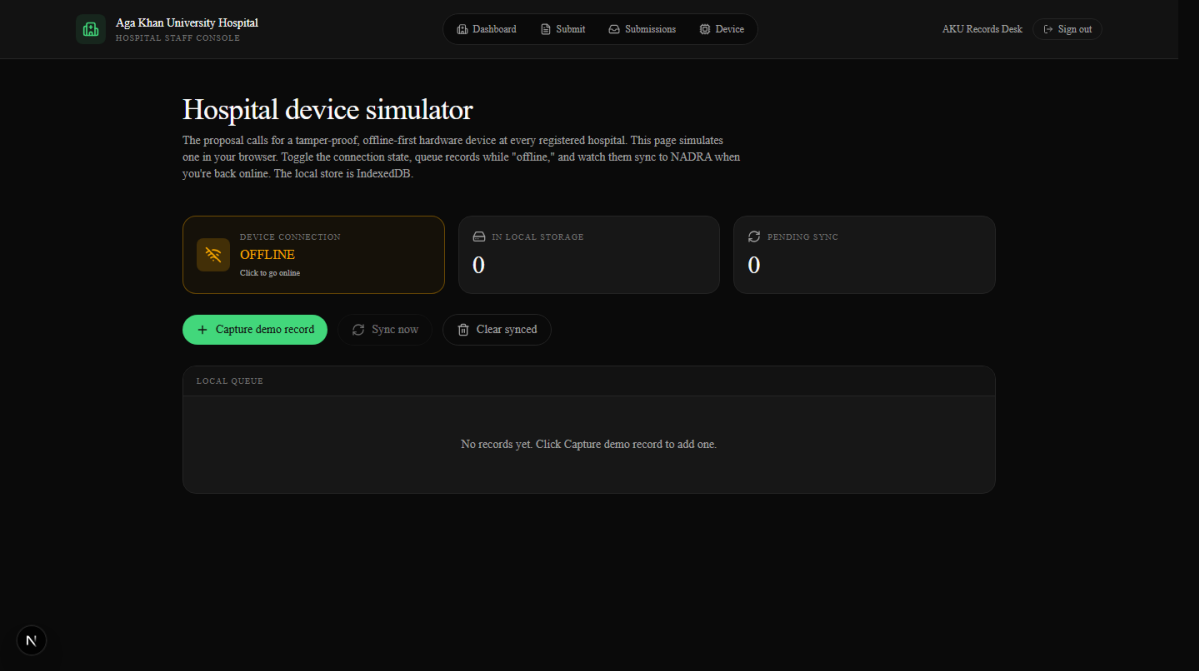


Figure 11.9 — Device simulator — IndexedDB-backed offline queue with manual online/offline toggle.

12. Conclusion

NBRPTS demonstrates that every concept in CS2013 — entities, relationships, normalization, constraints, joins, aggregates, GROUP BY, HAVING, triggers, transactions — has a direct, working expression when applied to a real problem. The system is deployed live, the schema is in 3NF, and every query in the rubric runs against real data.

The codebase is open and reproducible: clone the repository, run `supabase db push`, run `pnpm dev`, and the same screens shown in §11 will load in your browser, backed by your own copy of the database. Every line of SQL — every CREATE TABLE, every CHECK, every trigger — is exactly as it is documented in this report.

Live URLs

- Application: <https://nbrpts.vercel.app>
- Source: <https://github.com/ruufaayl/nbrpts>
- Demo accounts (password demo1234): `aku@nbrpts.demo` (hospital_staff), `aisha@nbrpts.demo` (nadra_officer), `admin@nbrpts.demo` (admin).

Tools used

- PostgreSQL 15 (via Supabase free tier, ap-southeast-1).
- Next.js 16 + React 19 (App Router, Server Components, Server Actions).
- Tailwind CSS v4, Framer Motion, react-flow + dagre.
- Supabase Auth (email + password, JWT cookies, `@supabase/ssr`).
- Vercel free tier for hosting; GitHub for source control; pnpm for package management.